

Table 2

HTTP request – sent by client GET method with HTTP headers only	<i>GET / HTTP/1.1</i> <i>Host: www.example.com</i> <i>User-Agent: curl/7.65.0</i> <i>Accept: */*</i>
HTTP response – sent by server Has HTTP headers and body	<i>HTTP/1.1 200 OK</i> <i>Accept-Ranges: bytes</i> <i>Cache-Control: max-age=604800</i> <i>Content-Type: text/html; charset=UTF-8</i> <i>Date: Sat, 11 Jan 2020 06:30:35 GMT</i> <i>Etag: "3147526947+ident"</i> <i>Expires: Sat, 18 Jan 2020 06:30:35 GMT</i> <i>Last-Modified: Thu, 17 Oct 2019 07:18:26 GMT</i> <i>Server: ECS (sjc/4E73)</i> <i>Vary: Accept-Encoding</i> <i>X-Cache: HIT</i> <i>Content-Length: 1256</i> <!doctype html> <html> <head> <title>Example Domain</title> <meta charset="utf-8" /> ...

method that indicates the type of service required by the client. The most popular is the GET method, which is a request for information. The server answers with a HTTP response that either provides the information or returns an error code. As HTTP is pure text, it is best to just look at the HTTP request-response pair to get a quick idea of the protocol (Table 2).

An HTTP message consists of a header and an optional body. In the example in Table 2, the GET request does not have a body, but the GET response has one.

SSL: As seen in the previous sections, the data transported on DNS and HTTP protocols is unencrypted. This implies that if the data can be captured by a third party, it is readily available for viewing and interpretation. Second, not only is the data available to the third party, it is possible for this party to even modify in-flight data for its own benefit. Thus, these protocols do not have the properties of:

1. Confidentiality, and hence are susceptible to data viewing by a passive third party.
2. Integrity, and hence are susceptible to data modification by an active third party.

 **Note:** Another property, called Authenticity, is also not provided by DNS and HTTP. SSL provides mechanisms for ensuring Authenticity also.

To address the above, Netscape defined the SSL protocol, which was

further enhanced and standardised by IETF as TLS. The SSL protocol has two phases: the Handshake phase and the Record phase. During the Handshake, the client and server negotiate a session key, which is used to encrypt the actual data that is transferred during the Record phase of the protocol. Then, the data in the Record protocol is opaque to a passive third party. Also, SSL incorporates data signatures such that any modification to the data can be detected by the other party. Thus, when HTTP is transported over SSL (and the resultant protocol is the very popular HTTPS), it is reasonably safe from third party attackers.

A quick recap

We started with understanding the phrase protocol stack in the context of computer networking. Then, we saw that ‘HTTP/DNS, TCP/UDP IP’ was the protocol stack for the early Internet. Later, this stack evolved first to HTTP/1.1, and then added an end-to-end security protocol, SSL. In the last decade, the Internet stack evolved more rapidly with the definition of new versions of protocols at various layers in the stack, including:

- HTTP2, HTTP3 and DNS-over-HTTPS at the application layer
- TLS 1.3 at the security
- QUIC at the transport layer

Then, we took a quick look at the format of the DNS, HTTP and SSL protocols. In the next article in this series, we will discuss the HTTP2, TLS 1.3 and DNS-over-HTTPS protocols. 

References

- [1] DNS overview: <https://www.cloudflare.com/learning/dns/what-is-dns/>
- [2] HTTP introduction: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview>
- [3] SSL introduction: <https://www.cisco.com/c/en/us/support/docs/security-vpn/secure-socket-layer-ssl/116181-technote-product-00.html>

By: Sriram Gorti

The author is an avid fan of the simplicity and ease of programming with Python, and tinkers with it on weekends. On weekdays, he maintains and extends production code for a cloud web security solution.